Planificador distribuido de tareas MCP con DAG ponderado y prioridades de camino crítico

Adrian Compean Sepúlveda Daniel Camacho Contreras
Luis Contla Ramírez Rodrigo Cortés Pacheco
Equipo 1

***Resumen*—Se implementó un planificador de tareas MCP que construye un DAG ponderado y distribuye trabajo con MPI por prioridad de camino restante. La corrección del scheduler se distingue de la incertidumbre del grafo inferido. Se comparan ejecución secuencial, FIFO y prioridad crítica. En 54 ejecuciones controladas se conservaron los resultados; las 27 verificaciones de consultas alcanzaron exactitud, completitud y consistencia de 1.00. La carga sensible a prioridades redujo el tiempo 31.8% frente al baseline secuencial. También se verificaron MPI y MCP reales. La evaluación autenticada de Jev y el despliegue multinodo siguen pendientes.**

I. Parte I Análisis algorítmico

A. Q1 Definición motivación y supuestos

Los agentes pueden secuenciar herramientas independientes. Se busca reducir su latencia conservando resultados, precedencias e insumos.

La entrada es un JSON con n tareas, herramientas, argumentos, referencias a resultados, dependencias explícitas, artefactos producidos y consumidos, recursos y costos positivos. El análisis obtiene un DAG $G = (V, E)$ con pesos $\hat{w}_v$ y razones auditables para cada arista. Dados p workers, se producen asignación $a(v)$, inicios s_v, resultados y estados terminales. Con duraciones reales w_v, se minimiza

$$C_{\max} = \max_{v \in V}(s_v + w_v), \tag{1}$$

sujeto a $s_v \geq 0$, $s_v \geq s_u + w_u$ para $(u, v) \in E$, una tarea por worker y exclusión de accesos incompatibles a recursos. Lectura–escritura y escritura–escritura agregan precedencias; dos lectores pueden coincidir.

El modelo es $P \mid prec \mid C_{\max}$: procesadores idénticos y tareas indivisibles, sin expropiación ni migración. Rank 0 coordina y no cuenta como worker. La implementación añade comunicación, serialización y procesos aislados. Se asumen herramientas de confianza, datos serializables y metadatos que describan insumos y efectos compartidos. Ante fallos se exigen resultados parciales explícitos.

El problema es NP-difícil: con $E = \varnothing$, $p = 2$ y pesos enteros, decidir un makespan igual a la mitad del trabajo contiene PARTITION. La heurística de listas permite liberación dinámica sin buscar el óptimo combinatorio. Jev aporta decisiones estructuradas [1]; su probabilidad no demuestra que el DAG contenga todas las dependencias reales.

Algoritmo 1 Planificación dinámica
1. Validar DAG; calcular b e indegrees.
2. Encolar nodos con indegree cero.
3. Mientras haya tareas listas o activas:
   a. Con un worker libre, extraer la mejor tarea, resolver argumentos y enviar trabajo.
   b. Recibir la siguiente respuesta disponible.
   c. Si tuvo éxito, guardar resultado y reducir indegrees; encolar nuevos hijos listos.
   d. Si falla, reintentar solo si es idempotente y queda presupuesto; si no, omitir descendientes.
4. Detener workers; guardar reporte e historial.

Fig. 1. Liberación por eventos, sin barreras entre niveles completos.

B. Q2 Algoritmo central y corrección

Se combinan Kahn [2], camino restante y despacho al siguiente worker libre. Se agregan aristas por `depends_on`, referencias, artefactos y conflictos. La línea base heurística reconoce menciones exactas de artefactos en descripciones. Jev consulta precedencias entre pares y costos por intervalo, sin eliminar relaciones declaradas. Probabilidad seleccionada menor que 0.75 bloquea una dependencia; todo ciclo se rechaza antes de ejecutar.

En orden topológico inverso se calcula

$$b(v) = \hat{w}_v + \max_{x:(v,x)\in E} b(x), \quad \max \varnothing = 0. \tag{2}$$

Se elige mayor $b(v)$ y se desempata por prioridad explícita y posición en el JSON. FIFO usa orden de ingreso. Los pesos se congelan por ejecución.

Invariantes: I1, el indegree residual cuenta padres sin éxito; I2, solo se despachan tareas listas y cada worker tiene un intento activo; I3, las referencias usan resultados exitosos; I4, un fallo definitivo impide despachar descendientes. I1 vale inicialmente por construcción. Cada éxito reduce una vez sus aristas; encolar al llegar a cero preserva I1 e I2. Resolver referencias después preserva I3. Marcar descendientes antes del siguiente despacho preserva I4. Por inducción sobre eventos se respeta el DAG aceptado y sus exclusiones de recursos.

Todo subgrafo pendiente de un DAG finito tiene un nodo mínimo. Si las tareas terminan o alcanzan timeout, los reintentos son finitos y el transporte opera, se alcanzan estados terminales. La pérdida de un rank aborta el job: no se promete continuidad. La contribución integra dependencias auditables y ejecución MCP sobre MPI. Listas y camino crítico son

técnicas conocidas; no se propone una nueva garantía de optimalidad.

C. Q3 Complejidad garantías y límites

Con $m = |E|$, las adyacencias ocupan $O(n + m)$. Kahn con heap, prioridades y despacho cuestan $O((n + m) \log n)$ sin fallos. Calcular b después de ordenar cuesta $O(n + m)$; los reintentos agregan trabajo acotado. El análisis completo no es lineal: comparar pares para conflictos cuesta hasta $O(n^2 r)$ con r recursos por tarea, además de procesar descripciones. Jev formula $O(n^2)$ preguntas, almacenadas antes de dividirse en lotes; su espacio también es cuadrático.

Con B bytes de resultados retenidos y $b_{\max}$ como mayor mensaje activo, el estado de ejecución ocupa aproximadamente $O(n+m+B+pb_{\max})$, más memoria de herramientas. Rank 0 retiene resultados y puede ser cuello de botella. Sin reintentos hay n envíos de trabajo y n respuestas, más control. Un modelo es

$$T_{\text{comm}} \approx 2n\alpha + \beta D, \tag{3}$$

donde α es latencia, β tiempo por byte y D los bytes transmitidos, incluidas copias de insumos. Abrir una sesión MCP por tarea añade negociación [3], [4].

Con procesadores idénticos, herramientas exitosas y comunicación nula, una lista que no deja workers ociosos cuando hay trabajo listo satisface la cota clásica [5]:

$$C_{\text{LS}} \leq \frac{W}{p} + \left(1 - \frac{1}{p}\right) L \leq \left(2 - \frac{1}{p}\right) C^*. \tag{4}$$

Aquí $W = \sum_v w_v$, L es el camino más largo en duraciones reales y C^* el óptimo. La justificación sigue hacia atrás una cadena que explica la última finalización: fuera de sus intervalos todos los procesadores están ocupados y la cadena mide a lo sumo L. Esto da la primera desigualdad; $W/p, L \leq C^*$ dan la segunda. La garantía no incorpora costos MPI, heterogeneidad, reintentos ni aristas equivocadas. Los pesos predichos fijan prioridades; $\max b(v)$ no certifica un límite inferior del tiempo real. Se reportan speedup $S_p = T_1/T_p$ y eficiencia $\eta_p = S_p/p$.

D. Q4 Robustez escalabilidad y calidad

Se comparan dos alternativas: secuencial topológico con un worker, baseline sin optimización, y FIFO entre tareas listas. FIFO y prioridad crítica comparten transporte y dos workers en la evaluación controlada. No hay superioridad universal: con costos errados, FIFO puede empatar o superar a la prioridad crítica.

Se rechazan IDs repetidos, referencias desconocidas, ciclos, artefactos sin productor y pesos inválidos. El aislamiento permite timeout; MCP añade cancelación y hasta cinco segundos para cerrar la sesión. Los reintentos requieren idempotencia. Un fallo omite descendientes y conserva ramas independientes. La admisión por memoria sigue planeada; no se afirma protección frente a todo agotamiento de recursos.

La exactitud compara diez comprobaciones numéricas y estructurales por consulta contra referencias independientes del baseline, con tolerancia 10^{-9}. La completitud cuenta nueve campos requeridos. La corrección de precedencia exige que cada padre termine antes del primer intento del hijo. La consistencia compara SHA-256 del JSON canónico de resultados, excluyendo tiempos y recursos. Se evalúan herramientas y composición deterministas, no la calidad semántica de prosa libre de un LLM.

E. Q5 Limitaciones y próximos pasos

Las principales limitaciones son pesos poco sensibles al tamaño de entrada, inferencia cuadrática y datos centralizados. Se proponen:

Costos condicionados y ubicación. Predecir duración por bytes, herramienta y tipo de nodo; incorporar comunicación y finalización temprana, como motiva HEFT [6]. Medir error de predicción, makespan y bytes frente a la política actual en nodos heterogéneos.

Inferencia selectiva verificable. Indexar productores y recursos, consultar solo pares ambiguos y procesar preguntas en flujo. Medir precisión y recall de aristas contra DAGs revisados, tokens y latencia. Una predicción plausible no demuestra independencia.

Admisión y recuperación. Limitar concurrencia según memoria, transmitir referencias a datos grandes y guardar checkpoints. Evaluar RSS agregado, tiempo de recuperación y duplicados con fallos inyectados. Estas mejoras no se presentan como implementadas.

II. Parte II Reporte del proyecto

A. Panorama y arquitectura

El caso de uso es un agente que obtiene datos, los limpia, calcula estadísticas, detecta anomalías, produce un histograma y compone un resumen. Se busca menor latencia conservando todos los insumos y la respuesta de referencia. El límite inicial es p; se observa CPU y RSS individual, sin imponer aún cuotas de memoria.

Plan JSON $\rightarrow$ análisis $\rightarrow$ DAG ponderado
$\rightarrow$ rank 0 $\rightleftarrows$ workers MPI
$\rightarrow$ herramientas MCP $\rightarrow$ resultados e historial.

Rank 0 resuelve referencias y recibe con `ANY_SOURCE`. Cada worker ejecuta un proceso aislado. El cliente MCP inicializa la sesión stdio, invoca la herramienta, valida su estado y retorna resultados estructurados. Los workers no necesitan el grafo completo.

B. M1 Algoritmo central implementado

Se implementaron ambas políticas, conflictos, reintentos, timeout, exportación DOT y reportes JSON/HTML. SQLite aprende $\hat{w}_{\text{nueva}} = 0.3w_{\text{observado}} + 0.7\hat{w}_{\text{anterior}}$. Los pesos explícitos prevalecen sobre historial, intervalo Jev y heurística. Solo se aprende de éxitos y no se cambian prioridades a mitad del ensayo.

El plan original de siete tareas completó MPI con rank 0 y cuatro workers. Cuatro herramientas MCP se ejecutaron con rank 0 y tres workers, conservando el resultado local. Se probaron Python 3.12.4, mpi4py 4.1.2, MPICH 5.0.2 y MCP SDK 1.30.0. Se verificó stdio y varios procesos en una computadora; no se afirma validación multinodo.

TABLE I
EVALUACIÓN LOCAL. MEDIANAS DE TRES REPETICIONES, EN SEGUNDOS. FIFO Y CRÍTICA USAN DOS WORKERS; SECUENCIAL, UNO.

Carga	Tareas	Sec.	FIFO	Crítica
Consulta uniforme	5	0.586	0.439	0.487
Consulta constante	5	0.557	0.436	0.435
Consulta con extremo	5	0.535	0.417	0.421
Rama crítica	4	1.636	1.372	1.116
Abanico pequeño	9	1.041	0.683	0.576
Abanico mayor	25	2.902	1.508	1.522

TABLE II
MÉTRICAS DEL BENCHMARK FINAL.

Dimensión	Medición
Eficiencia	Tiempo, CPU, tokens, RSS agregado, bytes y costo con tarifa fechada
Calidad	Exactitud, corrección del DAG y completitud
Consistencia	Acuerdo de hashes y decisiones repetidas
Determinismo	Configuración fija; separar resultados, orden y variación Jev
Escalabilidad	Tiempo y eficiencia por carga y workers

La carga con rama crítica ahorró 31.8% de tiempo frente al secuencial y 18.6% frente a FIFO. Las consultas pequeñas no muestran dominio consistente sobre FIFO (Tabla I). La carga de 25 tareas alcanzó speedup 1.91 y eficiencia 95.3% con dos workers. Su CPU acumulado de tareas fue 1.988 s secuencial frente a 2.017 s con prioridad crítica: reducir latencia no demuestra ahorrar trabajo computacional ni energía.

C. M2 Robustez y evaluación determinista implementadas

Se fijaron tres consultas en `experiments/golden.json`: $[1, 2, 3, 4, 5]$, $[7, 7, 7, 7]$ y $[10, 20, 30, 40, 100]$. Sus referencias se calcularon directamente: estadística poblacional, anomalías con $|z| > 1.5$ y cinco bins incluyendo el extremo superior. La tercera tiene media 40, mediana 30, desviación $\sqrt{1000}$, extremo en índice 4 y frecuencias $[2, 2, 0, 0, 1]$.

Los 27 ensayos de consultas alcanzaron exactitud, completitud y corrección de precedencia 1.00, con hashes iguales entre métodos y repeticiones. La carga de prioridad y dos abanicos completan 54 ejecuciones, manteniendo valores, herramientas, pesos e intérprete, sin historial. Se registraron mediana, mínimo, máximo, CPU, RSS y hashes; los tiempos no son deterministas.

El proveedor fue heurístico, sin LLM ni muestreo: tokens remotos y costo de API fueron cero; energía y costo del hardware no se midieron. El mayor RSS individual fue 25.88 MiB; excluye memoria agregada y servidores MCP hijos. Hay 36 pruebas automatizadas de contrato, análisis, ejecución, FIFO y MCP, incluyendo cancelación. El sistema operativo puede cambiar recepción y asignación física: se exige igualdad de resultados para herramientas puras, no cronogramas idénticos.

Jev se probó con respuestas controladas, sin consulta autenticada. La evaluación pendiente fijará un ID inmutable de modelo y archivará solicitudes y respuestas, repitiendo diez análisis por plan contra aristas etiquetadas. No se inventarán parámetros de temperatura o semilla que la API no expone. La variabilidad del modelo se separará de la del scheduler y de las herramientas.

D. M3 Calidad y asignación de recursos planeadas

La próxima versión minimizará $C_{\max}$ y, secundariamente, CPU y costo remoto, con exactitud y completitud 1.00 en consultas deterministas, cero violaciones de precedencia y acuerdo de hashes 1.00. Para inferencia se propone precisión y recall de aristas al menos 0.99 en planes revisados. Son umbrales de aceptación, no resultados medidos. Una relación incierta detendrá el plan o exigirá una dependencia explícita validada.

Se seleccionará el menor p que cumpla latencia y presupuesto de memoria. Los ensayos MPI iniciales muestran por qué: las medianas de tres repeticiones fueron 2.256 s con dos workers y 2.472 s con cuatro, frente a 3.674 s secuenciales. Hubo dispersión y carga de fondo no aislada. Son mediciones exploratorias, no evidencia de escalamiento monotónico; el benchmark final aislará estos factores.

E. M4 Benchmark y sistema final planeados

Se conservará coordinador–workers, con admisión por memoria, artefactos por referencia y sesiones MCP persistentes. Se comparará secuencial, FIFO, prioridad crítica y una variante de HEFT con comunicación y heterogeneidad. Kahn, listas y camino crítico son las bases clásicas elegidas; su correspondencia exacta con el temario del curso debe confirmarse.

La matriz prevista contiene 8, 32, 128 y 512 tareas, distinta anchura y densidad, varios tamaños de entrada y $p \in \{1, 2, 4, 8\}$. Habrá calentamiento, diez o más repeticiones, orden aleatorizado, máquina sin carga ajena y versiones congeladas. Se reportarán mediana, percentil 95 e intervalo de confianza del speedup, verificando referencias y hashes por ensayo. Se distinguirán pruebas multinodo de las locales.

Se aceptará ahorro solo si se mantienen los umbrales y mejora la métrica objetivo. El costo de API sumará tokens por tarifa real, sin confundirlo con energía ni CPU. La memoria total exigirá muestrear ranks, procesos de tarea y servidores, complementando el RSS individual.

F. Repositorio y hoja de ruta

Se entregan código, planes, esquema, servidor MCP, pruebas, referencias fijas, evaluación, resultados y fuentes del informe. Las versiones están congeladas. El contenedor no se presenta como verificado.

M1 está implementado y verificado en MPI. M2 tiene evidencia determinista y robustez; la variabilidad Jev está pendiente. M3 requiere costos condicionados, cuotas y selección de p. M4 requiere comparadores heterogéneos, aislamiento y validación multinodo. El orden será cerrar aristas de referencia y fijar versión Jev, implementar admisión y memoria total, comparar cargas ampliadas y reproducir en varios nodos.

Referencias

[1] TypeSafe, System One API y Choice. https://docs.typesafe.ai/api/system-one. Consulta: 23 de octubre de 2026.

[2] A. B. Kahn, Topological sorting of large networks, *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962. doi: 10.1145/368996.369025.

[3] Model Context Protocol, Python SDK, v1.x. https://github.com/modelcontextprotocol/python-sdk/tree/v1.x. Consulta: 23 de octubre de 2026.

[4] mpi4py, Tutorial e Installation, versión 4.1.2. https://mpi4py.github.io/mpi4py/stable/html/tutorial.html. Consulta: 23 de octubre de 2026.

[5] R. L. Graham, Bounds on multiprocessing timing anomalies, *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969. doi: 10.1137/0117039.

[6] H. Topcuoglu, S. Hariri y M.-Y. Wu, Performance-effective and low-complexity task scheduling for heterogeneous computing, *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002. doi: 10.1109/71.993206.

Appendix A
Reproducción

Desde la carpeta del proyecto:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-lock.txt
python -m unittest discover -s tests -v
python scripts/evaluate.py
python scripts/benchmark_mpi.py
```

Para MCP real distribuido:

```
mpiexec -n 4 python -m midterm run \
 examples/mcp_plan.json --backend mpi \
 --no-history --output results/mpi_mcp.json
```

La evaluación local está en `results/evaluation.json`; las referencias, en `experiments/golden.json`. Los tiempos excluyen análisis Jev y arranque MPI, pero incluyen procesos por tarea y sesiones MCP cuando corresponde. La plataforma fue macOS arm64, con el mismo intérprete entre métodos. `results/scaling.json` y su CSV identifican doce ensayos MPI exploratorios. `results/mpi.json` verifica siete tareas en cinco ranks; `results/mpi_mcp.json`, herramientas MCP en cuatro ranks. No se midieron energía ni varios nodos.

Appendix B
Formato y entrega

Las fuentes usan `IEEEtran`, modo `conference`, dos columnas. Las referencias y estos apéndices se separan del cuerpo. Se conserva `ID` en el archivo para que los autores añadan sus identificadores. Q1–Q5 y M1–M4 distinguen implementación verificada de trabajo planeado.